15.051 Mathematical programming, HW2

Erich Trieschman

2025.10.02

1. **3.9 Necessary and sufficient conditions for global optimum**

   Given: Linear problem in standard form, $\min_x c^T x \quad \text{s.t. } Ax = b,\ x \geq 0$ and x^* optimal. $B, N = B^C$ the sets of basic and nonbasic indices of the optimal basis for x^*. $I \in N$ such that $\bar{c}_i = 0$ for $i \in I$

   (a) **Show:** I empty $\Rightarrow x^*$ only optimal solution

   Contradiction: Suppose $\exists\ y^* \neq x^*$ s.t., $c^T y^* = c^T x^*$, a distinct second optimal solution. We can define search direction $d = y^* - x^*$. Since y^*, x^* optimal BFS, we have

   $$0 = Ay^* - Ax^* = Ad = \sum_{i \in B} A_{B(i)} d_{B(i)} + \sum_{j \in N} A_{N(j)} d_{N(j)} = Bd_B + Nd_N$$

   $$d_B = -B^{-1} N d_N$$

   Now the cost improvement vector is

   $$\begin{aligned} 0 = c^T(y^* - x^*) = c^T d &= c_B^T d_B + c_N^T d_N \\ &= c_N^T d_N - c_B^T B^{-1} N d_N = \left(c_N^T - c_B^T B^{-1} N\right) d_N \\ &:= \bar{c}_N^T d_N \end{aligned}$$

   Now from what we know,

   $$\begin{aligned} x_N = 0 &\Longrightarrow d_N = y_N^* - x_N^* = y_N^* \\ y_N^* \text{ optimal solution} &\Longrightarrow y_N^* \geq 0 \Longrightarrow d_N \geq 0 \\ x^* \text{ optimal} &\Longrightarrow \bar{c}_N \geq 0 \\ (I = \varnothing) \cup (\bar{c}_N \geq 0) &\Longrightarrow \bar{c}_N > 0 \\ (\bar{c}_N > 0) \cup (d_N \leq 0) \cup (\bar{c}_N^T d_N = 0) &\Longrightarrow d_N = 0 \end{aligned}$$

   So for optimal solution $y^* \neq x^*$, there is no change in direction d_N. But we also know that $Nd_N = 0 \Longrightarrow Nd_N = -Bd_B = 0$. Since B invertible, it has a unique solution to $-Bd_B = 0 \Longrightarrow d_B = \mathbf{0}$.

   Hence $d = [d_B, d_N] = \mathbf{0} \Longrightarrow 0 = y^* - x^* \Longrightarrow y^* = x^*$. Contradiction

(b) **Show:** x^* only optimal solution $\iff$ $\sum_{i \in I} x_i = 0$ in the below problem

$$\begin{aligned} \max \quad & \sum_{i \in I} x_i \\ \text{s.t.} \quad & Ax = b \\ & x_i = 0 \text{ for } i \in N \setminus I \\ & x_i \geq 0 \text{ for } i \in B \cup I \end{aligned}$$

i. $\implies$: **(Contradiction)** Let y be any feasible point s.t. $\sum_{i \in I} y_i > 0$. Since it is a feasible solution, we know $Ay = b$ and $y \geq 0$.

Define $d = y - x^*$. $Ad = 0 \implies d$ is a feasible direction

From part a), we have $c^T y - c^T x^* = \bar{c}_N^T d_N$. Since x^* optimal $\implies c^T y \geq c^T x^* \Rightarrow \bar{c}_N^T d_N \geq 0$

Since $x_N = 0 \implies d_N = y_N$. Now, from our supposition, $\exists\ i \in I$ such that $y_i > 0 \implies d_i > 0$. By definition of I, the corresponding $\bar{c}_i = 0$. And to be feasible in our axillary problem (i.e., $x_i = 0$ for $i \in N \setminus I$), we must have $y_{N \setminus I} = d_{N \setminus I} = 0$.

Therefore, $0 = \bar{c}_N^T d_N = c^T y - c^T x^* \implies c^T y = c^T x^*$. Contradiction to x^* being the only optimal solution

ii. $\impliedby$: **(Contrapositive)** Suppose x^* is not the only optimal solution, then we show solution to the auxillary problem is $\sum_{i \in I} x_i > 0$.

Given:

- x^* not unique optimal $\implies \exists$ feasible $y \neq x^*$ such that $c^T y = c^T x^*$
- Part a) by contrapositive: x^* not unique optimal $\implies I \neq \emptyset \implies \exists\ i \in I$ such that $\bar{c}_i = 0$

For $\theta d = y - x^*$ we know we can construct feasible y from search direction d with:

- $d_i = 1$
- $d_j = 0$ for $j \in I \setminus \{i\}$
- $d_B = -B^{-1} A_i$

Such d is a valid search direction since

$$Ad = \sum_{j \in B} A_{B(j)} d_{B(j)} + \sum_{j \in N} A_{N(j)} d_{N(j)} = B d_B + A_i = -B B^{-1} A_i + A_i = \mathbf{0}$$

so

$$Ay = A(x + \theta d) = Ax + \theta Ad = Ax = b$$

and y is feasible for chosen $\theta = -\min_{\{i \in B | d_i > 0\}} \frac{x_i}{d_i}$, since $y = x + \theta d \geq 0$ for such a θ.

Now we have feasible y with

$$\begin{aligned} 0 &= c^T(y - x^*) = \theta c^T d \\ &= (c_N^T - c_B^T B^{-1}) N d_n = \bar{c}_N^T d_N \qquad \text{(see derivation in part a)} \end{aligned}$$

since $\theta d_i > 0$ and $\theta d_{N \setminus \{i\}} = 0$ by construction $\implies \bar{c}_i = 0 \implies \{i\} \subseteq I \implies \sum_{i \in I} y_i \geq \theta d_i > 0$

2. **3.12 Simplex implementation**

Given:

$$\begin{aligned} \min \quad & -2x_1 - x_2 \\ \text{s.t.} \quad & x_1 - x_2 \leq 2 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

(a) Convert to standard form and construct BFS at $(x_1, x_2) = (0, 0)$

$$\begin{aligned} \min \quad & -2x_1 - x_2 \\ \text{s.t.} \quad & x_1 - x_2 + x_3 = 2 \\ & x_1 + x_2 + x_4 = 6 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{aligned}$$

BFS: $(x_1, x_2, x_3, x_4) = (0, 0, 2, 6)$

(b) Carry out Tableau implementation of simplex, starting at BFS from part a

Construct the initial Tableau matrix using

$$B = B^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad A = \begin{bmatrix} 1 & -1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 2 \\ 6 \end{bmatrix} \quad c = \begin{bmatrix} -2 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

Solve for quantities

$$\begin{aligned} -c_B^T B^{-1} b = 0 \qquad & \text{(since } c_B = \mathbf{0}\text{)} \\ c^T - c_B^T B^{-1} A = c^T \qquad & \text{(since } c_B = \mathbf{0}\text{)} \\ B^{-1} b = b \qquad & \text{(since } B^{-1} = I\text{)} \\ B^{-1} A = A \qquad & \text{(since } B^{-1} = I\text{)} \end{aligned}$$

And we get Tableau

$$\begin{bmatrix} -c_B^T B^{-1} b & c^T - c_B^T B^{-1} A \\ B^{-1} b & B^{-1} A \end{bmatrix} = \begin{bmatrix} 0 & c^T \\ b & A \end{bmatrix} = \begin{bmatrix} 0 & -2 & -1 & 0 & 0 \\ 2 & 1 & -1 & 1 & 0 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$$

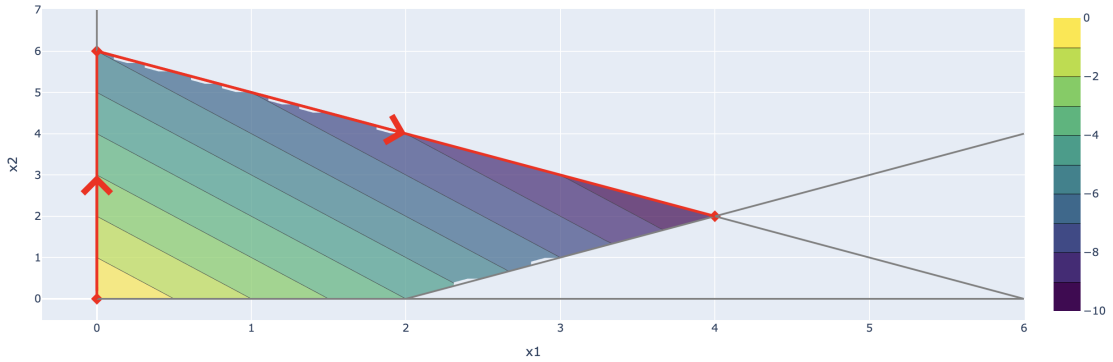
And we have

Step	**Iteration 1**	**Iteration 2**
1. Choose pivot column, l, based on $\bar{c}^T$	$\begin{bmatrix} 0 & -2 & -1 & 0 & 0 \\ 2 & 1 & -1 & 1 & 0 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 6 & -1 & 0 & 0 & 1 \\ 8 & 2 & 0 & 1 & 1 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$
2. Choose pivot row, i, using ratio test	$\begin{bmatrix} 0 & -2 & -1 & 0 & 0 \\ 2 & 1 & -1 & 1 & 0 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 6 & -1 & 0 & 0 & 1 \\ 8 & 2 & 0 & 1 & 1 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$
3. Apply row operations to get $u_l = e_l$	$\begin{bmatrix} 0 & -2 & -1 & 0 & 0 \\ 8 & 2 & 0 & 1 & 1 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 6 & -1 & 0 & 0 & 1 \\ 4 & 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 2 & 0 & 1 & -\frac{1}{2} & \frac{1}{2} \end{bmatrix}$
4. Apply row operations to set $\bar{c}_l = 0$	$\begin{bmatrix} 6 & -1 & 0 & 0 & 1 \\ 8 & 2 & 0 & 1 & 1 \\ 6 & 1 & 1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 10 & 0 & 0 & \frac{1}{2} & \frac{3}{2} \\ 4 & 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 2 & 0 & 1 & -\frac{1}{2} & \frac{1}{2} \end{bmatrix}$

$$x^* = \begin{bmatrix} 4 \\ 2 \\ 0 \\ 0 \end{bmatrix} \geq 0, \quad Ax = \begin{bmatrix} 1 & -1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \end{bmatrix} = b, \quad c^T x = \begin{bmatrix} -2 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \\ 0 \\ 0 \end{bmatrix} = -10$$

(c) Graphical representation of the problem and show simplex algorithm path

The Tableau simplex algorithm takes us on path $(x_1, x_2, x_3, x_4) = (0, 0, 2, 6) \longrightarrow (0, 6, 8, 0) \longrightarrow (4, 2, 0, 0)$. In $\mathbb{R}^2$ without the slack variables this path is $(x_1, x_2) = (0, 0) \longrightarrow (0, 6) \longrightarrow (4, 2)$. Plotted[1] below in $\mathbb{R}^2$:



[1] I used Google Gemini for help making this graphic

3. **3.25 Simplex with upper bound constraint**

Given:

$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \quad A \in \mathbb{R}^{m \times n} \text{ with linearly independent rows} \\ & 0 \leq x \leq u, \quad u \in \mathbb{R}^n > 0 \end{aligned}$$

(a) **Given:** Let $\{B(1), \ldots, B(m)\} = B$ be the set of indices of linearly independent columns of A. Partition index set $j \notin \{B(1), \ldots B(m)\} \in B^C$ into L, U. Set $x_j = 0 \quad \forall j \in L$ and set $x_j = u_j \quad \forall j \in U$. Then solve $Ax = b \quad \forall x_i$ for $i \in B$

i. **Show:** x is a basic solution

To show x is a basic solution we need to show
(A) All equality constraints are active: We start by writing $A = [A_B, A_L, A_U]$ and $x = [x_B, \mathbf{0}_L, u_U]^T$ in block form. Since x_B is constructed as the solution to $Ax = b$,

$$\begin{aligned} Ax &= A_B x_B + A_L \mathbf{0} + A_U x_U = b \\ x_B &= A_B^{-1}(b - A_U x_U) \end{aligned}$$

Then we have

$$\begin{aligned} Ax &= A_B x_B + A_L \mathbf{0} + A_U x_U \\ &= A_B A_B^{-1}(b - A_U x_U) + A_U x_U = b \end{aligned}$$

So all equality constraints are active

(B) Of the constraints that are active at x, n are linearly independent. We show this is true by construction. Consider all active constraints in matrix form:

$$\begin{bmatrix} A_B & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & I_L & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & I_U \end{bmatrix} \begin{bmatrix} x_B \\ \mathbf{0}_L \\ u_U \end{bmatrix} = \begin{bmatrix} b \\ \mathbf{0}_L \\ u_U \end{bmatrix}$$

Evaluated at x, the rows of the LHS matrix of active constraints is block diagonal with diagonal blocks A_B, I_L, I_U. These rows are linearly independent if each block is linearly independent. Indeed, block A_B is linearly independent by construction (m linearly independent columns in an $m \times m$ matrix), as are identity matrices $I_L \in \mathbb{R}^{|L| \times |L|}$ and $I_U \in \mathbb{R}^{|U| \times |U|}$.

ii. **Also show:** x nondegenerate $\iff x_i \neq 0$ and $x_i \neq u_i \quad \forall i \in B$
(**Recall:** A basic solution is degenerate if $> n$ (not necessarily linearly independent) constraints are active at x)

$\Longrightarrow$: **(Contradiction)** Suppose $\exists i \in B$ such that $x_i = 0$ or $x_i = u_i$. Then we can freely add one more active constraint to the ones identified in part (A), either $e_i^T x = 0$ or $e_i^T x = u_i$. This extends the number of equality constraints to $n + 1 > n$ and hence x is degenerate, which is a contradiction.

$\Longleftarrow$: Suppose $x_i \neq 0$ and $x_i \neq u_i \quad \forall i \in B$. If so, then the only active constraints are the ones defined in part (A): $Ax = b$, $x_L = 0$, and $x_U = u_U$. These are exactly n constraints and hence x is nondegenerate.

(b) **Given:** x is a basic feasible solution. Let j such that $x_j = 0$ and $\bar{c}_j < 0$. We adjust the entering and leaving variables: $x_j + \theta^*$ and $x_B - \theta^* B^{-1} A_j$.
Want: The largest value of θ^* such that updated x is feasible? How are new basic columns determined?

We aim to update x to $x + \theta d$, where $\theta > 0$ and search direction d is defined with $d_j = 1$, $d_i = -B^{-1}A_j \ \forall i \in B$, and $d_k = 0$ otherwise. In a standard form polyhedron with only lower bound, $x \geq 0$, the maximum feasible step size is

$$\theta^* = \min_{\{i | d_i < 0, i \in B\}} \left(-\frac{x_i}{d_i} \right)$$

This however doesn't guarantee this step size will be feasible since, $\forall i \mid d_i > 0$ this step size will increase x_i, possibly with $x_i - \theta d_i > u_i$, violating our upper bound constraint. We therefore have to further control our stepsize with

$$\theta^* = \min \left[\min_{\{i | d_i < 0, i \in B\}} \left(-\frac{x_i}{d_i} \right), \quad \min_{\{i | d_i > 0, i \in B\}} \left(\frac{u_i - x_i}{d_i} \right) \right]$$

The exiting variable is the *argmin* of this problem.

(c) **Given:** x is a basic feasible solution. Let j such that $x_j = u_j$ and $\bar{c}_j > 0$. We want to adjust the entering and leaving variables: $x_j - \theta^*$ and $x_B + \theta^* B^{-1} A_j$.
Want: The largest value of θ^* such that updated x is feasible? How are new basic columns determined?

By convention that $\theta > 0$, we want to define our search vector, d with $d_j = -1$, $d_i = B^{-1}A_j \ \forall i \in B$, and $d_k = 0$ otherwise. Again we require $0 \leq x_i \leq u_i$. $d_i > 0 \Rightarrow x_i$ increases and $d_i < 0 \Rightarrow x$ decreases. So with this definition of our search direction, d, we arrive at the same definition for θ^*:

$$\theta^* = \min \left[\min_{\{i | d_i < 0, i \in B\}} \left(-\frac{x_i}{d_i} \right), \quad \min_{\{i | d_i > 0, i \in B\}} \left(\frac{u_i - x_i}{d_i} \right) \right]$$

Again, the exiting variable is the *argmin* of this problem.

(d) **Given:** Every basic feasible solution is nondegenerate.
Show: the cost strictly decreases with every iteration and the method terminates.

Intuitively, $\bar{c}_j$ tells us how much our objective changes when we change x_j. When $\bar{c}_j < 0$, the objective decreases when we increase x_j and when $\bar{c}_j > 0$, the objective increases when we increase x_j. So in our upper+lower bound problem, any $(\bar{c}_j < 0,\ x_j = 0)$ or any $(\bar{c}_j > 0,\ x_j = u_j)$ are candidate entering variables.

Parts (b) and (c) above show us how to construct feasible search directions and step sizes, hence we can define a simplex-like algorithm with a few modifications:

- We consider entering variables $(\bar{c}_j < 0,\ x_j = 0)$ and $(\bar{c}_j > 0,\ x_j = u_j)$
- When the entering variable is $x_j = u_j$, we take the negative of the simplex search direction
- We use the feasible step sizes defined in parts (b) and (c)
- In every step, our objective decreases by $\theta^* \bar{c}_j$

Since we're given that every basic feasible solution is nondegenerate, this algorithm does not revisit any bases and since there are a finite number of bases, the algorithm will terminate in a finite number of steps.

4. **3.27 Maximize number of positive components**

(a) **Want:** $x \in \mathbb{R}^n$ that satisfies $Ax = 0,\ x \geq 0$ with the maximum number of positive components
Show: This problem can be achieved with

$$\max \sum_i y_i \quad s.t. \quad A(z+y) = 0,\ y \leq 1,\ z, y \geq 0$$

Feasible (z,y): Begin with any feasible solution x to $Ax = 0,\ x \geq 0$. We can trivially write $x = z + y,\ z, y \geq 0,\ y \leq 1$, demonstrating that for any feasible x, we can construct feasible (z, y) for our problem.

Feasible directions: Our objective value increases in directions that increase y_i (up to 1). Increasing our feasible solution by ty_i increases our objective by t, but increases the LHS of our constraint by tA_i:

$$A(z + y + te_i) = A(z + y) + tA_i = tA_i$$

We can make a feasible step with te_i if we can also find offsetting Δz such that $(z - \Delta z) \geq 0,\ A\Delta z = tA_i$. Then

$$A(z - \Delta z + y + te_i) = A(z + y) - A\Delta z + tA_i = -tA_i + tA_i = 0$$

Now for any λ satisfying $A\lambda = 0$, we see A_i can be written as a linear combination of columns $A_{j \neq i}$.

$$A\lambda = 0 \iff \sum_j \lambda_j A_j = 0 \iff \lambda_i A_i = -\sum_{j \neq i} \lambda_j A_j$$

When $\lambda_i \neq 0$, we have

$$A_i = -\sum_{j \neq i} \frac{\lambda_j}{\lambda_i} A_j$$

So long as $\exists \lambda$ such that $\lambda_i \neq 0$, then A_i can be written as some linear combination of the other columns of A. And in the context of the above problem, we can define

$$\Delta z_i = 0, \quad \Delta z_{j \neq i} = t\frac{\lambda_j}{\lambda_i}$$

$$t^* = \max\{t : t \geq 0,\ y_i + t \leq 1,\ z_j - t\frac{\lambda_j}{\lambda_i} \geq 0\ \forall j \neq i\}$$

To construct feasible direction $e_i - \Delta z$ and step size t^*.

Conic constraint: Note that our primal problem's constraints form a cone

$$x \in \mathcal{C} = \{x \mid Ax = 0, x \geq 0\}$$

which means that the feasible solutions are invariant to scaling. i.e.,

$$Ax = 0 \Rightarrow A(\alpha x) = \alpha Ax = 0 \quad \forall \alpha \in \mathbb{R}$$

Hence, can we always scale up $(z + y)$ so that $t^* = 1 - y_i$ for feasible index i. This means the optimal value to our problem will equal the maximum number of nonzero entries of x.

Maximum in finite iterations. To conclude this proof, observe that any step made in the feasible direction described above improves our objective, specifically by $t*$. Since our problem is a linear program, we are guaranteed to achieve the maximum in a finite number of steps.

(b) **Want:** $x \in \mathbb{R}^n$ that satisfies $Ax = b,\ x \geq 0$ with the maximum number of positive components.
Show: This problem can be achieved with a linear program.

Proposed linear program:

$$\begin{aligned} \max_{z,y,\theta} \quad & \sum_i y_i \\ s.t. \quad & A(z + y) = \theta b \\ & y \leq 1,\ z, y, \theta \geq 0 \end{aligned}$$

Feasible (z, y, θ)**:** Begin with any feasible solution x to $Ax = b,\ x \geq 0$. We can trivially select $z, y, \theta \geq 0,\ y \leq 1$ such that $\theta x = (z + y)$, so

$$A(z + y) = A(\theta x) = \theta Ax = \theta b$$

Demonstrating that for any feasible x, we can construct feasible (z, y, θ) for our problem.

Feasible directions: As in part (a), we can increase our objective function by t by increasing our feasible solution by te_i, so long as we can find offsetting shifts to stay feasible.

$$A(z - t\Delta z + y + te_i) = (\theta + t\Delta\theta)b, \text{ feasible if } \exists\ \Delta z, \Delta\theta \text{ s.t. } A_i = A\Delta z + \Delta\theta b$$

This is an equivalent problem to our original search for Δz in part (a), only lifted to $\mathbb{R}^{n+1}$. We can increase the value of y_i so long as we can find any such vector $[\Delta z, \Delta\theta]^T$ such that A_i can be written as a linear combination of the columns of $[A, b]$

Conic constraint: Our new equality constraint lifted into (x, θ) space is also a cone constraint. Specifically, if point (z, y, θ) is scaled by $\alpha > 0$, the new point is still feasible:

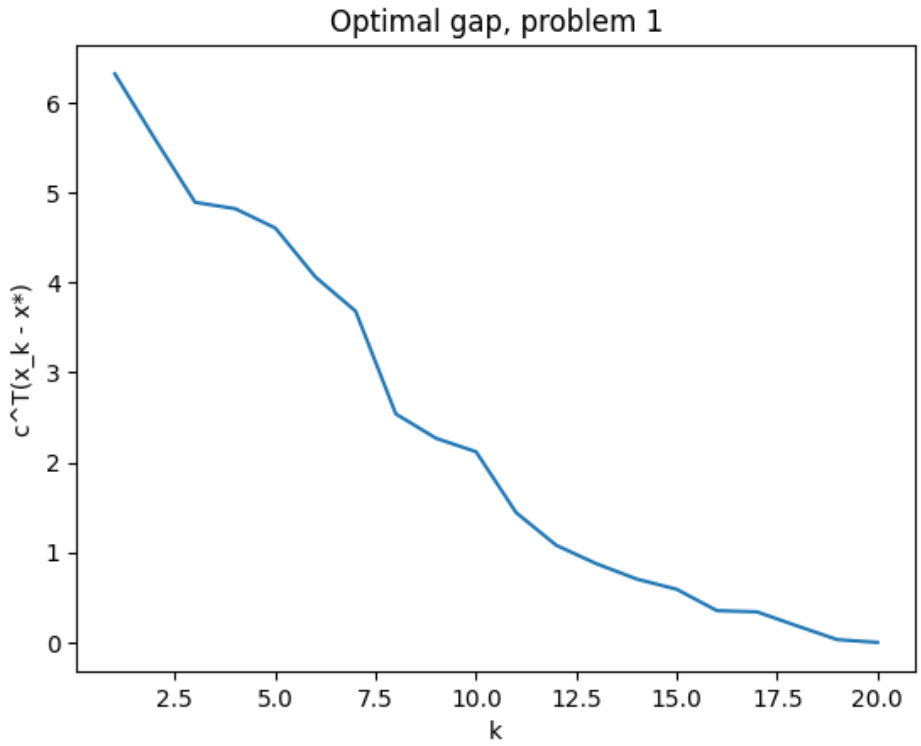
$$A(\alpha z + \alpha y) = \alpha A(z + y) = \alpha\theta b$$

Hence, as in part (a), any feasible point can be scaled up so that all positive y_i reach their upper bound of 1. Therefore, the optimal value of our linear program equals the maximum number of positive components of any feasible x satisfying $Ax = b,\ x \geq 0$.

5. **Simplex implementation and solutions**

Solutions.

```
==== problem 1 ====
---- part 1 ----
Phase 1 simplex - find BFS
        status:          optimal
        opt. value:          0.0
Phase 2 simplex - optimize
        status:         optimal
        opt. value:          4.0499
---- part 2 ----
```




```
==== problem 2 ====
---- part 1 ----
Phase 1 simplex - find BFS
        status:           optimal
        opt. value:         3.1735
Phase 2 simplex skipped - infeasible
        status:        infeasible
        opt. value:         None
---- part 2 ----
No iterations to show


==== problem 3 ====
---- part 1 ----
Phase 1 simplex - find BFS
        status:        optimal
        opt. value:         0.0
Phase 2 simplex - optimize
        status:        unbounded
        opt. value:         None
---- part 2 ----
No iterations to show
```

Code.

```python
import numpy as np
import os
from typing import Tuple

# read in data
dir = os.getcwd()
def get_data(problem):
    A = np.loadtxt(f"{dir}/data/hw2_data/{problem}/A.csv", delimiter=",")
    b = np.loadtxt(f"{dir}/data/hw2_data/{problem}/b.csv", delimiter=",")
    c = np.loadtxt(f"{dir}/data/hw2_data/{problem}/c.csv", delimiter=",")
    return A, b, c

# Simplex with a given BFS
def simplex_w_bfs(
        A, b, c, x_bfs, eps = 1e-10, max_iter = 200
    ) -> Tuple[str, np.ndarray | None, np.ndarray | None]:
    x_i = x_bfs
    x_is = [x_i.copy()]
    for i in range(max_iter):
        # -1- step 1: initialize variables
        B = x_i > 0
        A_B_inv = np.linalg.inv(A[:, B])
        u = A_B_inv @ A
        # -2- step 2: calculate cost difference
        c_bar = c - c[B] @ u
        c_bar[np.abs(c_bar) < eps] = 0
        # CHECK: return optimal if cbar > 0
        if np.all(c_bar >= 0):
            return "optimal", x_i, np.array(x_is)
        # -3- step 3: choose entering variable
        e = int(np.where(c_bar < 0)[0][0]) # first negative
        e = int(np.argmin(c_bar)) # most negative
        d = np.zeros(x_i.shape[0])
        d[e] = 1
        d[B] = -u[:, e]
        # CHECK: return unbounded if d >= 0
        if np.all(d >= 0):
            return "unbounded", None, None
        # -4- step 4: compute greedy step size
        S = (d < 0) & B
        theta = np.min(x_i[S] / -d[S])
        x_i += theta * d
        x_i[np.abs(x_i) < eps] = 0
        x_is += [x_i.copy()]
    return "max_iter", x_i, np.array(x_is)

# Two-phase simplex
def simplex_2phase(A, b, c) -> Tuple[str, np.ndarray | None, np.ndarray | None]:
    # -A- Initialize Phase 1
    A_p = np.concatenate([A, np.eye(A.shape[0])], axis=1)
    A_p[b < 0, :A.shape[1]] *= -1
    b_p = np.abs(b)
    c_p = np.concat([np.zeros(c.shape[0]), np.ones(b.shape[0])])
    xy_bfs = np.concat([np.zeros(A.shape[1]), b_p])
    # check:
    assert np.allclose(A_p @ xy_bfs, b_p)

    # -B- Phase 1
    print("Phase 1 simplex - find BFS")
    status, xy_opt, __ = simplex_w_bfs(A_p, b_p, c_p, xy_bfs)
    print(f"\tstatus:\t\t {status}")
    print(f"\topt. value:\t {np.round(c_p @ xy_opt, 4) if xy_opt is not None else None}")
    # check:
    if (c_p @ xy_opt) > 1e-5:
        print("Phase 2 simplex skipped - infeasible")
        print("\tstatus:\t\t infeasible")
        print("\topt. value:\t None")
        return "infeasible", None, None

    # -C- Phase 2
    x_bfs = xy_opt[:A.shape[1]]
    print("Phase 2 simplex - optimize")
    status, x_opt, x_is = simplex_w_bfs(A, b, c, x_bfs)
    print(f"\tstatus:\t\t {status}")
    print(f"\topt. value:\t {np.round(c @ x_opt, 4) if x_opt is not None else None}")
    return status, x_opt, x_is
```